Onboarding Guide: Making Your First Documentation Contribution in GitHub

Background

New technical writers, particularly those with 0-1 years of experience, often enter the field with strong writing skills but will have limited exposure to version control systems like GitHub. Landing a first job might require the use of GitHub for their first time to contribute to the team. Those transitioning from non-technical roles or with little experience often struggle with the multi-step workflow required to make even minor edits without disrupting the production environment.

Purpose

The purpose of this project is to standardize the contribution process through a structured, Web UI-based guide. This standard operating procedure (SOP) ensures junior writers can perform essential Jobs-to-be-Done with confidence and zero production issues. Aside from the how-to, this document will also help the writers understand the role of repositories and branches in a professional development workflow.

Introduction

Why GitHub

- **Collaboration**: Multiple writers can work on the same project simultaneously without overwriting each other's work.
- **History**: All changes and uploads are tracked. Any mistakes can be rolled back to the previous version.
- **Accountability**: Every update shows who created and approved the update, what was updated, when, and why it was updated.
- **Central Hub**: All documentation is stored in the repository allowing any authorized team member to access the most current files.
- **Simultaneous Updates**: Different sections of a project can be updated at once. Approved sections can be instantly published to a live web page.

Branch Role

- **Main Branch**: The live version used for production visible to the public or customers.
- **Working Branches**: A dedicated workspace where the updates are refined before moving to the main branch.

- **Features**: Allows for the creation of new sections or updates without risk of affecting the main version.
- **Isolation**: New versions with different layouts or updates can be tested before implementation.

Conditions

The following conditions must be met in order to complete this task:

- **Access**: User must have a GitHub account.
- **Permissions**: User must have "Write" or "Maintainer" access to the specific repository.
- **Environment**: A stable internet connection and access to a web browser (Web-UI).
- **Branch**: User must select the development branch to ensure no changes are made to the main branch prior to review and approval

Glossary

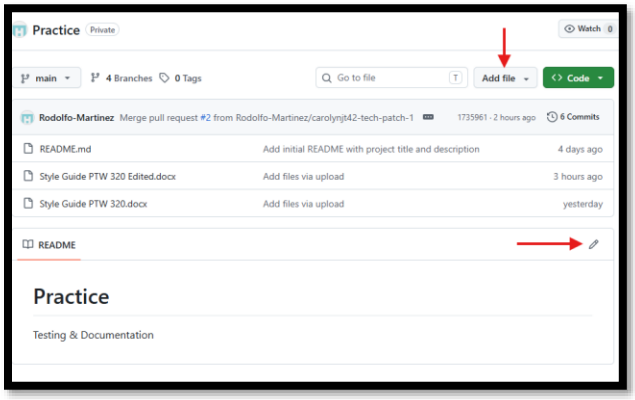
- **Repository ("Repo"):** Your project's master folder page where all files and history are stored.
- **Main Branch:** The live version, or production ready document(s) that everyone sees.
- **Working Branch:** Your "Drafting Sandbox" (can be named anything). A parallel space where you can safely edit, delete, or experiment without touching the official version.
- **Commit:** A snapshot of your changes or like a save point. A commit requires a message to record exactly what was updated and why.
- **Pull Request (PR):** A request for review. The last step before a reviewer approves your update or document for the live version.
- **Merge:** Once your PR is approved, merging permanently adds your changes into the Main branch. (not part of this SOP since you will not have approval access)
- **Merge Conflict:** This happens if you and a teammate edit the exact same line of the same file. GitHub stops the merge so you can manually pick which version to keep. Notify your admin if this happens. (Not part of this SOP since you will not have approval access.)
- **Sync:** A refresh button that updates your working branch with the latest changes to ensure you are working on the most recent version.
- **README:** It is the first file people see that explains what the project is, how to use it, and how to contribute.

Procedure

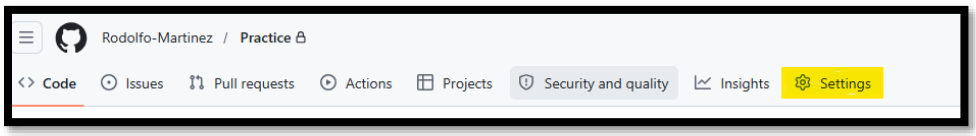
**Quick Note: If this is your first time using GitHub, your home screen will not have any repositories on the left side bar. You must be added as a collaborator to the project repository first. After the repository admin grants you permission, follow the steps below to make your first contribution.*

Verify Repository Permissions:

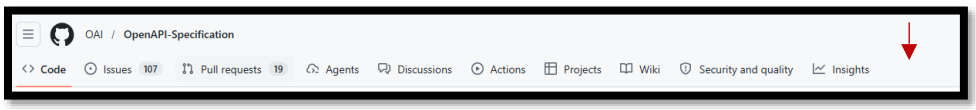
1. Confirm you have "Write" or "Maintainer" access to ensure you are not blocked from saving changes.
    a. Navigate to the repository homepage.
    b. Look for action button "**Add file**" (sometimes shown as a "**+**" sign) or the **settings tab**.
        - Note: A **Pencil** (edit) icon "✎" may appear if you click on a specific file like the README file.
        - Seeing these action buttons confirms you have "write" capabilities.



    c. You should also be able to see and click on the settings tab.

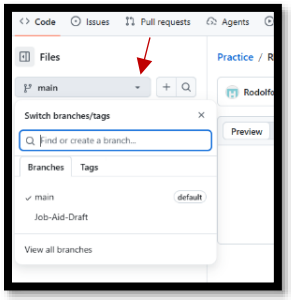


- A missing settings tab (and action buttons) means you have "Read-only" access. Please contact the repository admin to request permission

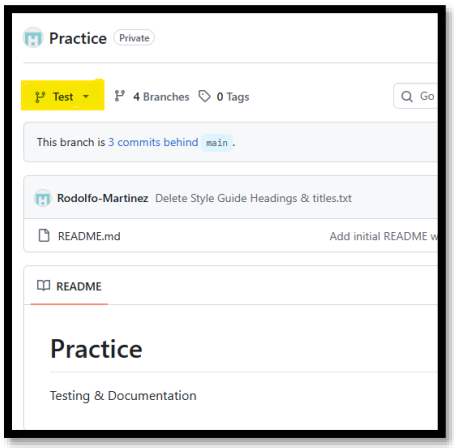


Select or Create a Working Branch

2. Switch to a Development Branch to avoid updating the **main** branch.
    a. Open the Branch selector: Click on the drop-down menu (usually labeled “Main”).



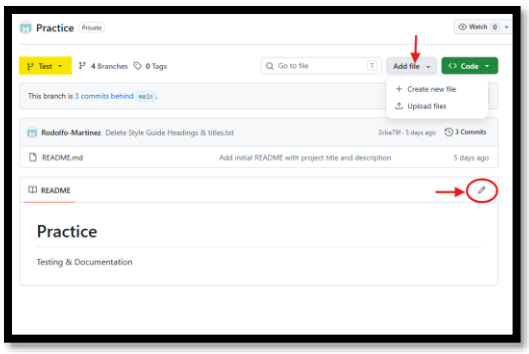
    b. Type in new branch name or select existing development branch if available.
    c. Confirm the dropdown is not the Main branch and you are in a development branch.



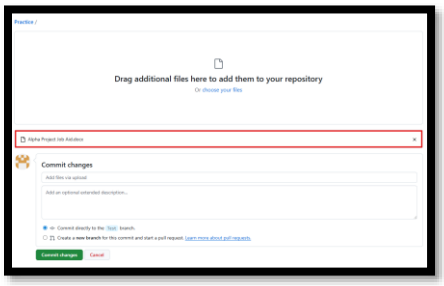
Critical: Do not edit or upload directly to the Main branch. All updates need approval before merging.

Upload or Edit via Web UI

3. Use the "Add file" or "Edit" (pencil icon) buttons to physically move content into the repo.
    a. Click on the “Add file” drop down.
    b. Select “Upload files” (or “+ Create new file” if applicable).

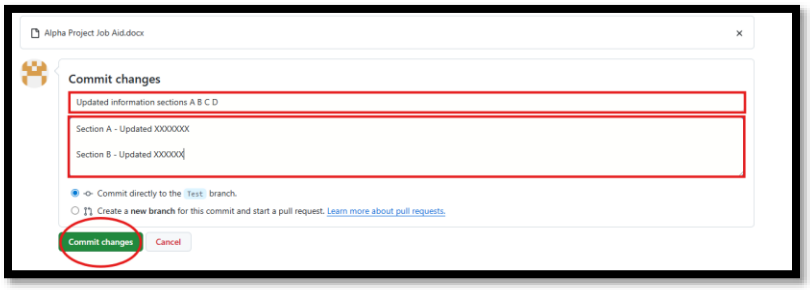


c. Choose your file or drag and drop document.



Write a Standardized Commit Message

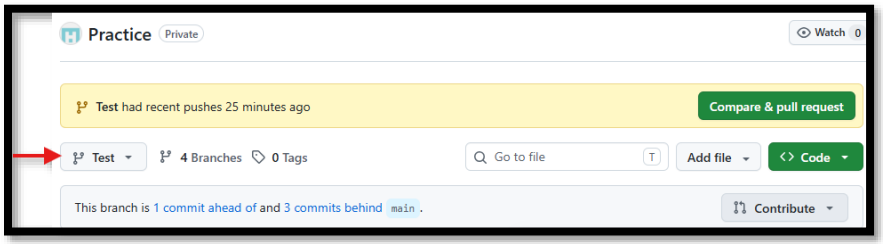
4. Before clicking **Commit Changes**, provide a clear message detailing why the update is being made so the team understands the change.
    a. **Add files via upload** (Required): In the top text box, provide a brief summary of your changes. Think of this as a subject line for an email.
    b. **Extended description** (Optional): Provide additional details about specific changes made. Think of this as the email body with more details.



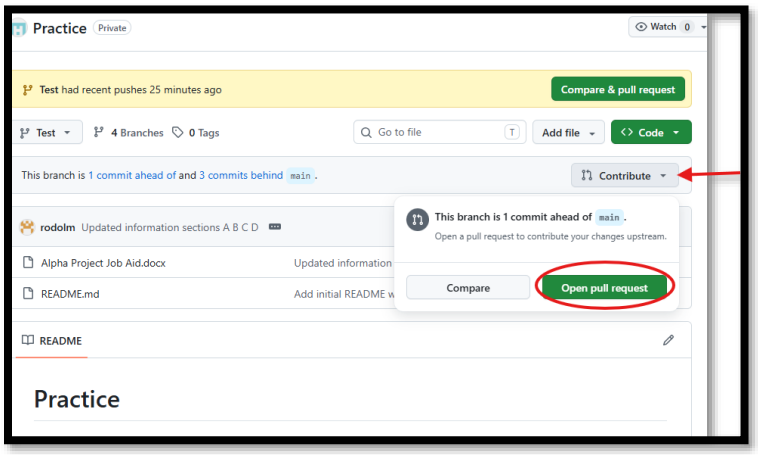
    c. **Commit**: Confirm update messages are clear, then click the **Commit Changes** button to save your file to the repository.

Submit a Pull Request (PR):

5. Propose the changes for review so a senior writer or developer can approve the merger.
    a. Check your branch. *If you are not in your working branch, your documents will not be visible.*

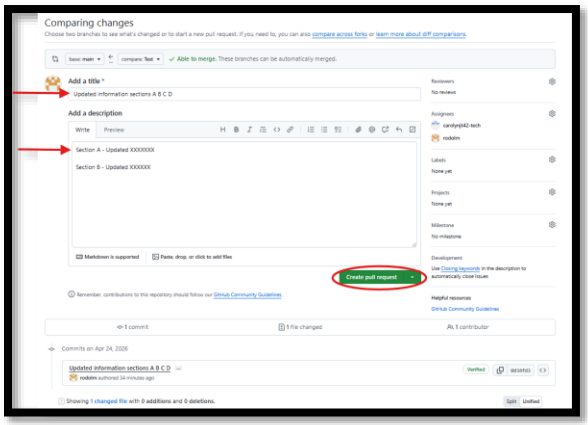


b. Open the **Pull Request**: Click the Contribute drop down button.

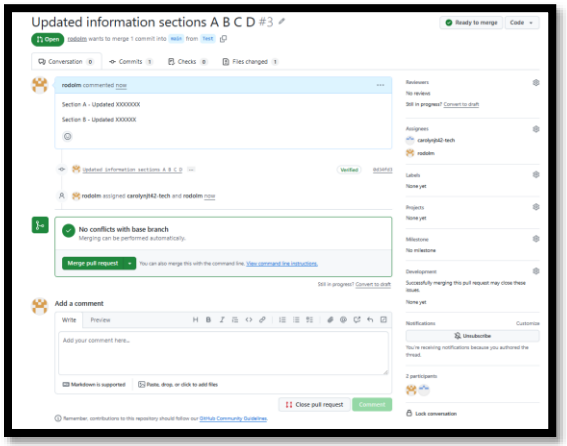


c. Select “Open pull request.”

d. Review and create: Review your changes on the screen. If message was not originally created, make sure to add the update message now. Click **Create pull request** when ready.



e. Wait for approval: Pull request submitted, awaiting merger upon approval.



Your part is complete! Now wait for the administrator to review the work and merge it into the main branch or provide feedback.